

Comprehensive Test Plan & Test Cases

Enterprise-Grade Algorithmic Trading System with Agentic AI Assistant

Table of Contents

- Comprehensive Test Plan & Test Cases
 - Enterprise-Grade Algorithmic Trading System with Agentic AI Assistant
 - Table of Contents
 - Introduction
 - Test Scope
 - Test Objectives
 - Test Strategy
 - Test Environment
 - Test Deliverables
 - Test Schedule
 - Roles and Responsibilities
 - Risks and Mitigations
 - Test Cases
 - Unit Testing
 - Integration Testing
 - System Testing
 - Performance Testing
 - Security Testing
 - Usability Testing
 - Compliance Testing
 - Appendices

Introduction

This Test Plan outlines the strategy, scope, and procedures for testing an enterprise-grade algorithmic trading system integrated with an AI chatbot and agentic RAFT (Retrieval-Augmented Fine-Tuning). The system is designed to provide advanced trading capabilities, real-time market data analysis, and intelligent user assistance through natural language processing. Given the complexity and critical nature of the system, a comprehensive testing approach is essential to ensure its functionality, performance, security, and usability meet the highest standards.

The test plan is intended for the development team, quality assurance (QA) engineers, project managers, and stakeholders involved in the system's development and deployment. It provides a structured framework for executing and managing the testing process, ensuring all components are thoroughly validated before production release.

Test Scope

The scope of testing encompasses all major components and functionalities of the system, including:

- **Trading Engine:** Order placement, execution, cancellation, and modification.
- **Data Feed Integration:** Real-time and historical market data retrieval from multiple sources.
- **AI Chatbot:** Natural language understanding, response accuracy, and integration with trading functions.
- **Agentic RAFT:** Information retrieval, fine-tuning capabilities, and integration with the chatbot.
- **User Authentication and Authorization:** Secure access control and role-based permissions.
- **Portfolio Management:** Real-time tracking of positions, P&L, and risk exposure.
- **Strategy Backtesting:** Simulation of trading strategies with historical data.
- **Risk Management:** Pre-trade risk checks, circuit breakers, and compliance with trading limits.

Additionally, the scope includes non-functional aspects such as performance, security, scalability, and usability.

Out of Scope:

- Third-party broker systems (e.g., Interactive Brokers), except for integration points.
- External data sources, beyond verifying the system's fallback mechanisms.

Test Objectives

The primary objectives of the test plan are to:

- Ensure the system meets all functional requirements as specified in the product requirements document (PRD) and technical specification document (TSD).
- Validate the system's performance under normal and peak load conditions.
- Confirm the system's security measures protect sensitive financial data and comply with regulatory standards.
- Verify the usability of the AI chatbot and user interface for a seamless user experience.
- Identify and mitigate any defects or risks before production deployment.

Test Strategy

The test strategy employs a multi-faceted approach to ensure comprehensive coverage:

- **Unit Testing:** Validate individual components (e.g., trading engine functions, AI modules) in isolation.
- **Integration Testing:** Verify interactions between components (e.g., trading engine and data feeds).
- **System Testing:** Test the entire system end-to-end, simulating real-world scenarios.
- **Performance Testing:** Assess the system's responsiveness, scalability, and stability under load.
- **Security Testing:** Identify vulnerabilities and ensure compliance with security standards.

- **Usability Testing:** Evaluate the user interface and AI chatbot for ease of use and effectiveness.
- **Compliance Testing:** Ensure adherence to financial regulations and industry standards.

Testing Tools:

- **Unit Testing:** pytest (Python), Jest (JavaScript)
- **Integration Testing:** Postman, custom Python scripts
- **System Testing:** Selenium, Cypress
- **Performance Testing:** Locust, JMeter
- **Security Testing:** OWASP ZAP, OpenVAS
- **Usability Testing:** User feedback sessions, heuristic evaluation

Test Environment

The test environment mirrors the production setup to ensure accurate results:

- **Hardware:** Cloud-based infrastructure (e.g., AWS EC2 instances) with scalable resources.
- **Software:**
 - **Operating System:** Ubuntu 20.04 LTS
 - **Database:** PostgreSQL with pgvector, Qdrant
 - **Web Server:** Nginx
 - **Application Server:** Gunicorn (Python), Node.js
 - **Containers:** Docker, Kubernetes for orchestration
- **Data:** Synthetic market data for testing, with fallback mechanisms configured.
- **Network:** Simulated network conditions to test latency and reliability.

Test Deliverables

The following deliverables will be produced during the testing process:

- Test Plan Document (this document)
- Test Cases and Scripts

- Test Execution Reports
- Defect Reports and Logs
- Performance Benchmark Reports
- Security Audit Reports
- Usability Test Results
- Final Test Summary Report

Test Schedule

The testing schedule is aligned with a 5-phase development plan:

- **Phase 1: Foundational Setup** - Week 1-2: Unit testing of core components.
- **Phase 2: API Bridge** - Week 3-4: Integration testing of API endpoints.
- **Phase 3: AI Assistant MVP** - Week 5-6: System testing of AI chatbot and RAFT.
- **Phase 4: Full User Experience** - Week 7-8: Usability and performance testing.
- **Phase 5: Advanced Features & Enterprise Readiness** - Week 9-10: Security and compliance testing.

Roles and Responsibilities

- **QA Lead:** Oversees the testing process, manages test plans, and coordinates with development.
- **QA Engineers:** Design and execute test cases, report defects, and verify fixes.
- **Developers:** Assist with unit testing, provide technical support for test environments.
- **Security Specialists:** Conduct security audits and penetration testing.
- **Usability Experts:** Facilitate user testing sessions and analyze feedback.

Risks and Mitigations

Potential risks and their mitigation strategies include:

- **Data Source Failures:** Implement and test fallback mechanisms for data feeds.
- **Performance Bottlenecks:** Conduct early load testing and optimize critical paths.
- **Security Vulnerabilities:** Perform regular security scans and code reviews.
- **AI Misinterpretations:** Use diverse test cases and user feedback to refine AI models.
- **Regulatory Non-Compliance:** Engage compliance experts for audits and reviews.

Test Cases

Below are detailed test cases for each testing type, covering all major system components.

Unit Testing

Component: Trading Engine

- **Test Case ID:** TC-TE-001
Description: Verify market order placement.
Steps:
 1. Authenticate as a trader.
 2. Submit a market order for 10 shares of AAPL.
Expected Result: Order is placed successfully, and confirmation is received within 1 second.
- **Test Case ID:** TC-TE-002
Description: Verify limit order placement with price condition.
Steps:
 1. Authenticate as a trader.
 2. Submit a limit order for 10 shares of AAPL at \$150.
Expected Result: Order is placed only if the price condition is met.
- **Test Case ID:** TC-TE-003
Description: Verify order cancellation.
Steps:

1. Place an order.
2. Cancel the order before execution.

Expected Result: Order is cancelled, and status is updated.

Component: Data Feed Integration

- **Test Case ID:** TC-DF-001

Description: Verify real-time data retrieval for stocks.

Steps:

1. Request real-time price for AAPL.

Expected Result: Latest price is returned within 100ms.

- **Test Case ID:** TC-DF-002

Description: Verify historical data retrieval for options.

Steps:

1. Request historical data for AAPL call option (strike 150, exp. 2024-01-19).

Expected Result: Data points from the specified period are returned accurately.

Component: AI Chatbot

- **Test Case ID:** TC-AI-001

Description: Verify natural language understanding for portfolio query.

Steps:

1. Ask, "What's my current portfolio value?"

Expected Result: Accurate portfolio value is returned within 2 seconds.

- **Test Case ID:** TC-AI-002

Description: Verify response to an invalid query.

Steps:

1. Ask, "What's the weather today?"

Expected Result: Polite refusal or redirection to relevant trading queries.

Integration Testing

Test Case ID: TC-INT-001

Description: Verify trading engine integration with data feeds.

Steps:

1. Place a market order for AAPL.
2. Verify that the latest price from the data feed is used for execution.

Expected Result: Order is executed at the correct market price.

Test Case ID: TC-INT-002

Description: Verify AI chatbot integration with trading functions.

Steps:

1. Ask the chatbot to place a limit order for 10 shares of AAPL at \$150.
2. Check if the order is placed correctly via the trading engine.

Expected Result: Order is placed and confirmed through the chatbot.

System Testing

Test Case ID: TC-SYS-001

Description: End-to-end test of placing and executing a trade via the chatbot.

Steps:

1. Log in as a trader.
2. Use the chatbot to place a market order for 10 shares of AAPL.
3. Verify execution and portfolio update.

Expected Result: Trade is executed, and portfolio reflects the new position.

Test Case ID: TC-SYS-002

Description: Test fallback mechanism for data sources.

Steps:

1. Simulate failure of primary data source (e.g., Yahoo Finance).
2. Request real-time data for AAPL.

Expected Result: System switches to the next source (e.g., Alpha Vantage) and returns data.

Performance Testing

Test Case ID: TC-PERF-001

Description: Measure system response time under normal load.

Steps:

1. Simulate 100 concurrent users placing orders.
2. Measure average response time for order placement.

Expected Result: Average response time <100ms.

Test Case ID: TC-PERF-002

Description: Test system scalability with increasing load.

Steps:

1. Gradually increase the number of concurrent users from 100 to 1000.
2. Monitor system performance and resource utilization.

Expected Result: System scales horizontally without degradation in performance.

Security Testing

Test Case ID: TC-SEC-001

Description: Verify authentication mechanism.

Steps:

1. Attempt to access protected endpoints without a token.
2. Attempt to use an invalid token.

Expected Result: Requests are denied with 401 Unauthorized .

Test Case ID: TC-SEC-002

Description: Test for SQL injection vulnerabilities.

Steps:

1. Inject malicious SQL code into input fields (e.g., login form).

Expected Result: System sanitizes inputs and prevents unauthorized access.

Usability Testing

Test Case ID: TC-USAB-001

Description: Evaluate chatbot response accuracy and relevance.

Steps:

1. Provide a set of common trading queries to the chatbot.
2. Rate the responses for accuracy and helpfulness.

Expected Result: 90% of responses are accurate and relevant.

Test Case ID: TC-USAB-002

Description: Test user interface for ease of navigation.

Steps:

1. Ask users to perform tasks (e.g., place an order, view portfolio).
2. Measure task completion time and user satisfaction.

Expected Result: Users complete tasks within expected timeframes with minimal errors.

Compliance Testing

Test Case ID: TC-COMP-001

Description: Verify adherence to GDPR for user data.

Steps:

1. Check if user data is stored with consent.
2. Verify data retention policies and user rights (e.g., right to be forgotten).

Expected Result: System complies with GDPR requirements.

Test Case ID: TC-COMP-002

Description: Ensure pre-trade risk checks are enforced.

Steps:

1. Attempt to place an order that exceeds account risk limits.

Expected Result: Order is rejected with an appropriate error message.

Appendices

- **Test Case Template:**
 - **ID:** Unique identifier
 - **Description:** Brief overview
 - **Preconditions:** Setup required
 - **Steps:** Detailed actions
 - **Expected Result:** Desired outcome
 - **Actual Result:** Observed outcome
 - **Status:** Pass/Fail
- **Defect Report Template:**

- **ID:** Unique identifier
- **Summary:** Brief description
- **Steps to Reproduce:** How to trigger the defect
- **Severity:** Impact level (Critical, High, Medium, Low)
- **Priority:** Urgency of fix (P1-P4)
- **Assigned To:** Responsible developer
- **Status:** Open, In Progress, Resolved, Closed

This Comprehensive Test Plan & Test Cases document ensures that the algorithmic trading system is rigorously tested across all critical dimensions, from functionality and performance to security and usability. By following this plan, the development team can identify and address issues early, ensuring a high-quality, reliable system ready for production deployment.